

Deadrun • 丧尸在你家楼下追你

一个周末 + Claude Code vibe 出来的 GPS 健身游戏：丧尸群刷新在你真实的街道上，朝你真实的 GPS 位置走来，唯一的操作是你的腿。

本指南深挖它的三层技术：GPS 球面几何 → 手搓 Web Mercator 瓦片地图 → 纯函数游戏引擎，全部实验在电脑上无手机跑通。

Expo / React Native

TypeScript 纯函数引擎

OpenStreetMap • CARTO 瓦片

MIT 开源

GitHub: larprober/Deadrun

? 疑问清单（每问一张卡片）

1 这是什么？一句话能说清吗？

把跑步变成求生游戏：地图是你家楼下的真实街道，怪物是 GPS 坐标，手柄是你的腿。

开局站在原地，第 1 波 2~4 只丧尸刷新在 **95~200 米外** 的随机方位，以 1.1~1.9 m/s 的速度 **直线走向你的 GPS 位置**。每 45 秒来一波更多更快的。被摸到扣一条命（共 2~3 条），命尽 = 跑输。补给箱（医疗包/肾上腺素/照明弹）故意扔在 **离你 70~140 米、且离丧尸群最远的方向**——绕路捡补给才是这个游戏真正的玩法。

它是推文 [om_patel5](#) 那个“丧尸追你”App 的 **功能等价开源孪生**（原 App 未公开名字与代码；Deadrun 用 MIT 协议把同一套机制完整开源，README 写到教程级详细）。

2 为什么不用 Google 地图 SDK，反而手搓地图？

这是本项目最"反直觉"的架构决策。Google Maps SDK 对 Android 免费，但**签发 API key 的前提是在 Google Cloud 项目里绑一张信用卡**——对一个周末的爱好项目来说，"要一张卡"就是不可逾越的门槛；而且无限制 key 泄漏后会被盗刷计费 API。

替代方案：**CARTO 的暗色底图瓦片**（数据来自 OpenStreetMap）。填一个表单、免费、**不需要信用卡**、每月 500 万瓦片；不填 key 也能玩，只是每张瓦片上斜印着 "API KEY REQUIRED" 水印。瓦片用 `react-native-svg` 逐张画在屏幕上，**没有 react-native-maps**，**没有任何原生地图依赖**，Expo Go 里扫码即玩。

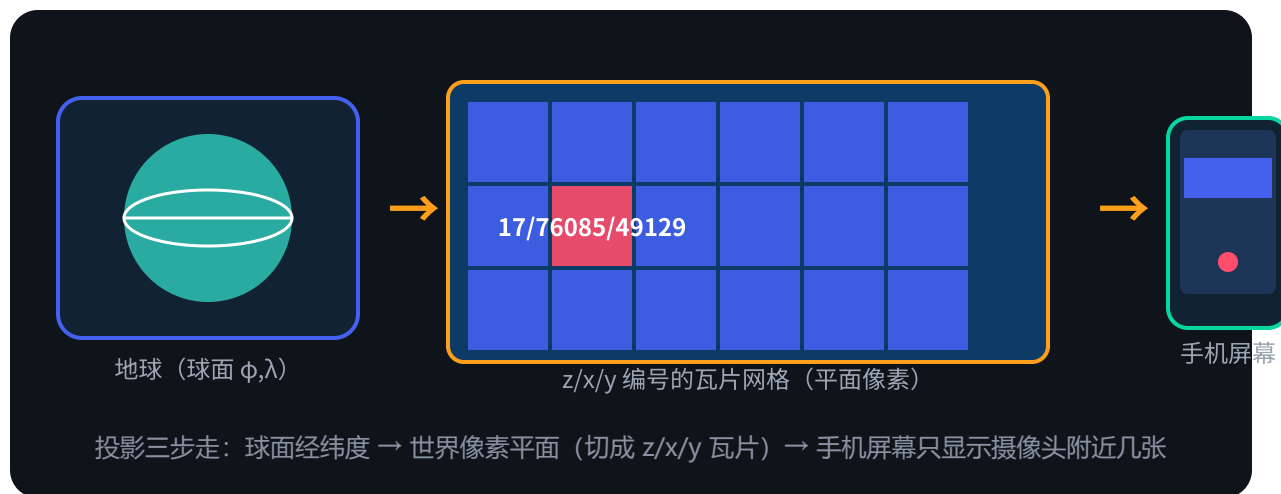
权衡：手搓地图 = 没有现成 SDK 的标注/手势/3D，只有"暗色底图 + 点与线"。对"跑步时瞥一眼"的场景，这恰恰是够用且最便宜的。

3 Web Mercator 投影：经纬度怎么变成屏幕像素？

手机屏幕是平面，地球是球面。**Web Mercator 投影就是两者之间的换算公式**：把经纬度映射到一个巨大的虚拟平面上（z17 zoom 时世界 = 131072×131072 像素），再切成 256×256 的瓦片，按 `z/x/y` 编号从服务器下载。

```
// src/tiles.ts — lat/lng → 世界像素 (z17)
x = (lng + 180) / 360 × 131072
y = (0.5 - ln((1+sinφ)/(1-sinφ)) / 4π) × 131072
```

已知bug的恐怖之处：符号写反或差一（off-by-one）不会报错，**症状是"地图显示错误的城市"**，而且要到户外跑步时才能发现。所以作者写了 `check-tiles.ts`：用教科书公式独立重算，再实际下载瓦片比对——本指南实验 2、3 复现了这个校验（241 倍字节差实证投影正确）。



4 瓦片预算：为什么地图旋转反而省瓦片？

每张瓦片 = 一次 HTTPS 请求 + 一张图，而手机同时还要跑 GPS、游戏引擎和音频。**"这次地图要拉多少张图"必须是个有上界的数**，作者把它钉死在 30 张以内并写进测试。

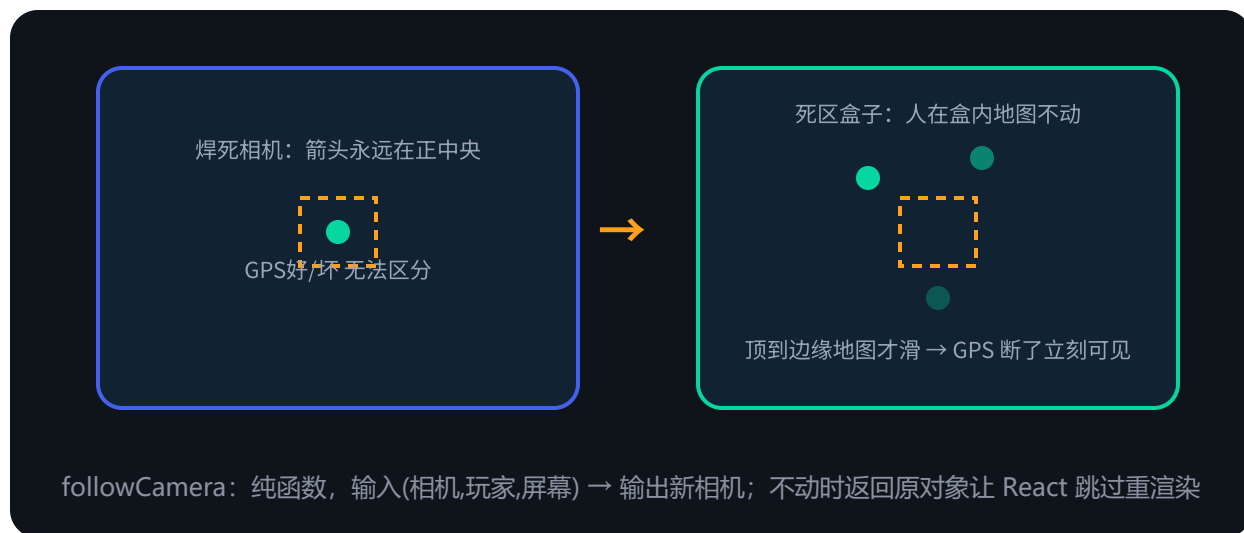
反常识点：本项目用**"退一级 zoom 的 @2x 高清瓦片"**（z16@2x = z17 的地面覆盖，但一张 512px 图顶 4 张 256px），清晰度相同、请求数省 4 倍。**旋转地图（course-up，永远朝前）**时屏幕的包围盒变成外接圆，需要 $5 \times 4 = 20$ 张，仍低于 30；若用普通 z18 瓦片会爆预算。

模式	瓦片窗口	张数	预算 30 张
north-up（北在上）	3×4	12	✓ 通过
course-up（行进方向在上）	5×4	20	✓ 通过

5 相机为什么不焊死在玩家身上？

大多数地图 App 把"我"钉在屏幕正中央。作者特意不这么做：屏幕中央留一个 **16%×12% 的"死区"盒子**，你在盒子里自由移动，只有顶到盒子边缘时地图才开始滑动跟上。

原因精妙：**焊死的相机让"GPS 完美"和"GPS 完全死了"长得一模一样**——两种情况下箭头都静止在正中央。让你在小盒子里移动，"我自己在屏幕上动"成了最直观的 GPS 健康指示器：fix 一断，你立刻看见自己不动了。



6 手机怎么知道"我朝哪"? GPS course 和罗盘该信谁?

这是两个不同的物理量：**GPS course (航向)**告诉你正在去哪，**罗盘 (磁力计)**告诉你**正在面对哪**。GPS 只有在移动时才算得出 course，且低于约 1.4 m/s (慢跑以下) 时基本是噪声；站着不动时 course 完全不存在，而且位置 fix 只在移动时才上报——此时读 fix 里的 heading 会**冻结在你最后移动那一刻的方向**，雷达和方向箭头全部悄悄指向错误方向。

融合规则 (`facingDeg`) 简单可靠：**速度 > 1.4 m/s 信 GPS course，否则信罗盘**；罗盘自己以 2° 节流独立上报。代码注释把这个 bug 的因果关系写得明明白白——这是"vibe coding 后人工把关"的范本。

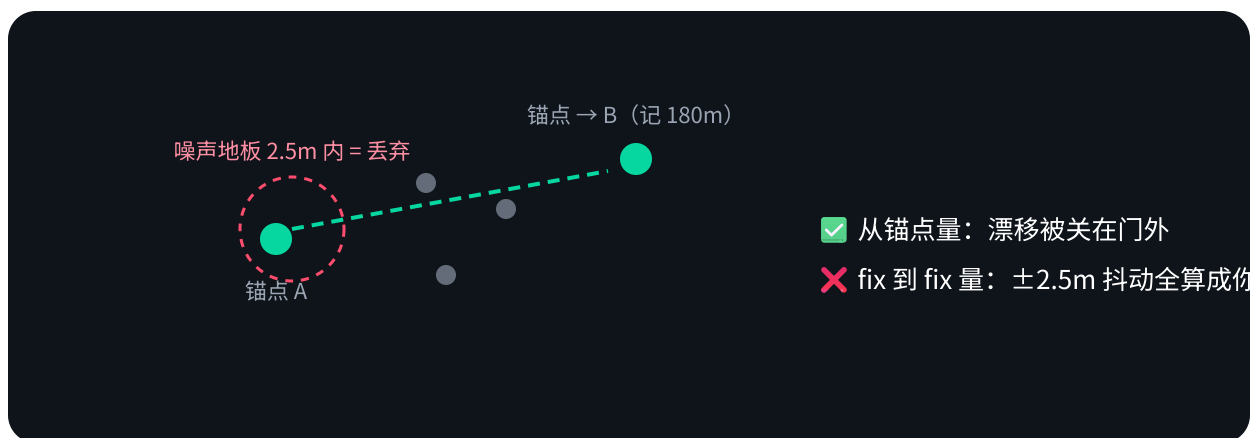
状态	速度	采用朝向
站着转身	0 m/s	罗盘 (GPS 无数据)
散步	1.0 m/s	罗盘 (course 是噪声)
跑步	3.0 m/s	GPS course (平滑准确)
罗盘还没就绪	—	退化用 course，再退化指正北

7 GPS 抖成筛子，游戏怎么知道我真的跑了 5 米？

民用 GPS 静止时会在原地抖出 ± 2.5 米甚至更大的"GPS 跳跃"。如果 fix 到 fix 直接算距离，就会给站着不动的玩家白送距离分。Deadrun 用了两个闸门：

① **锚点制 (anchor)**：距离永远从 **上一个"被认可的锚点"** 量起，而不是上一个 fix。新 fix 到锚点距离超过 $\max(2.5\text{m}, \text{精度} \times 0.5)$ 才记为真实移动并刷新锚点——移动小于噪声地板的步数被默默丢弃，同时避免了误差累积（和 SLAM 里"锚定"思想同源：每步都从上一步量，误差会滚雪球；从可信锚点量，误差有界）。

② **精度门槛**：fix 的 reported accuracy 差于 **40 米** 直接不用于计分和碰撞。



8 坐电动车刷分怎么办？反作弊怎么做才不误伤？

丧尸追的是人，不是车。规则：**速度持续超过 9 m/s (≈ 32 km/h) 累计满 12 秒，本局作废** (reason= `vehicle`)。细节在累加器：超速时按 1 倍时长累加，正常时按 **2 倍速衰减**——既给过红灯猛冲、GPS 抽风留缓冲，又让"匀速骑 15 公里"无法达标。配速 4:30/km 的精英跑者约 3.7 m/s，离 9 m/s 阈值很远，不会误伤。

另一个诚实的失败条件：GPS 信号丢失超过 **60 秒** 直接终局 (reason= `signal`) ——没有位置，游戏无法进行，这比假装活着更诚实。

9 丧尸的 AI 有多聪明？为什么要"预判拦截"？

别指望丧尸会寻路——它们只会**每 400ms 朝目标点直线走一小步**（engine tick），GPS 上报慢于 tick 时靠插值。但目标点的选择有讲究：

波次 < 4：目标 = 你的**最后已知位置**。你绕个弯它就追空，"秦王绕柱"有效。

波次 ≥ 4 且你在跑：目标 = **你前方 speed×lead 米的预判点**（lead 随波次升到 2.2 秒上限）——丧尸**抄你即将转弯的内角**，直线冲刺失效。这正对应"从第 3 波起，每波 1/3 的丧尸直接刷在你行进方向 ±38° 内"的伏击机制。

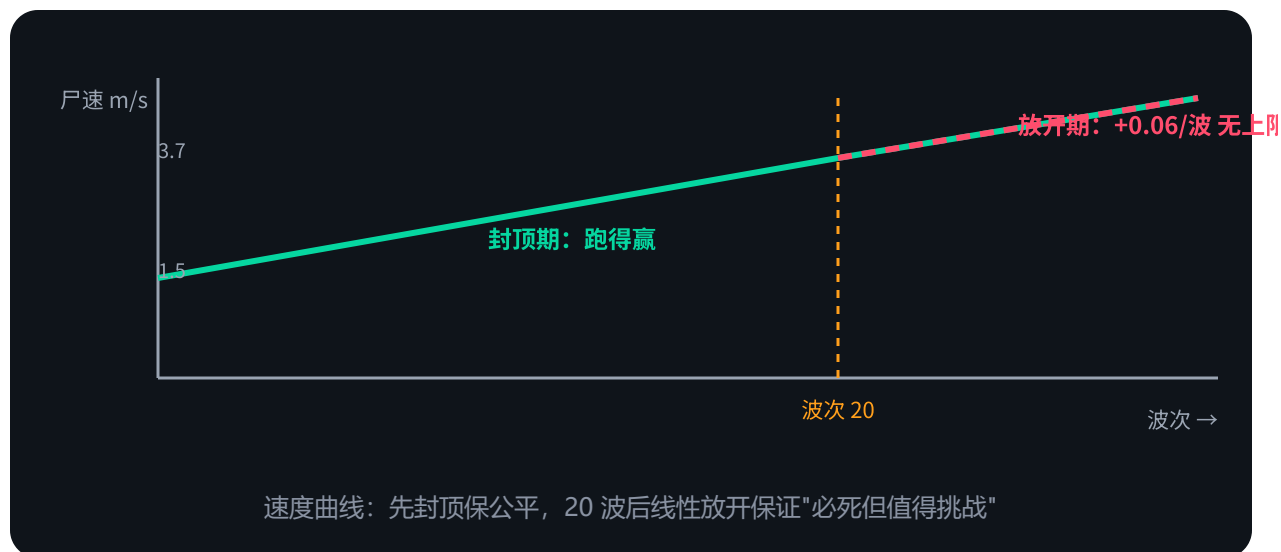
种类	速度系数	攻击半径	寿命	角色
walker 行尸	0.85×	11 m	无限	慢但永远跟着你
runner 跑尸	1.35×	9 m	70 秒后放弃	快速冲刺但会累
brute 巨尸	0.7×	20 m	无限	慢，但摸到你一下就没了

丧尸离你超过 420 米会失去气味（despawn），防止地图上无限堆积。

10 丧尸会无限加速吗？波次经济怎么设计才"死得其所"？

不会立刻无限加速——**中期（约波次 20 前）有速度天花板**（如 runner 难度 3.7 m/s），保证中段游戏公平、任何人都能靠策略活几分钟；**但过了第 20 波，每波再 +0.06 m/s 且无上限**——"没有任何配速能永远跑赢尸潮"，游戏终将以 caught 收场，你追求的不是赢，是**比上次活得更久**。

难度即经济：三种难度的 baseSpeed / 每波增速 / 初始数量 / 伏击起始波 / 分数倍率（1 / 1.5 / 2.25）全部集中在 `src/config.ts` 一个文件里，调平衡不用碰引擎逻辑。



11 为什么说引擎是"纯函数"的？这带来了什么超能力？

`engine.ts` 的每个函数都是纯函数：**输入旧状态 + 新 GPS fix，输出新状态 + 事件列表**，不读时钟、不碰网络、不存全局（随机数也是自己实现的种子 PRNG，不用 `Math.random`）。

超能力有三个：① **无头仿真**——整局游戏可以在电脑上用假 GPS 轨迹重放（本指南实验 1、4），调平衡不用出门；② **确定性**——同一种子复放同一局，bug 可复现；③ **时间旅行**——暂停恢复时 `shiftClocks` 把状态里所有绝对时间戳整体平移，尸潮不会在你暂停时瞬移。这也呼应知识库的 [[Checkpoint存档与持久执行]]：状态进、状态出，随时可以存档续跑。

12 一局的标准时间线长什么样？

以 runner 难度、seed 42、3.3 m/s 跑者为例（实验 1 实测）：开局 3 只丧尸在 126 米外 → 每 45 秒一波 → 第 5 波尸群 23 只、最近的只离你 18 米 → 第 6 波达到 26 只上限 → 一直坚持到 663 秒（11 分钟）后力竭被抓，得分 14656。站着不动的"雕像"玩家平均 2 分 16 秒就会被抓。

画像	波次	存活	距离	结局
站着不动	3.6	2m16s	0 m	全部 caught
散步 1.4 m/s	5.0	3m18s	273 m	全部 caught
慢跑 2.5 m/s	6.2	4m21s	650 m	全部 caught
跑者 3.3 m/s	14.4	10m33s	2085 m	全部 caught
跑者+捡补给	18.0	13m11s	2609 m	全部 caught
噩梦难度 4.5 m/s	14.2	10m13s	2757 m	全部 caught

9 种画像 × 5 种子平衡仿真实测输出（节选）——"你赢不了，只能比上次活得久"被数据证实



动手实验（全部命令实测跑通，无需手机）

实验 0 • 环境：Node 24 原生跑 TypeScript，零安装

操作：

```
node --version # 需 v22.6+（本机实测 v24.21.0）
```

目的：该项目的 `scripts/` 只依赖 Node 内置能力（fetch、类型剥离），不需要 Expo、不需要手机。**注意：**项目原本的 `npm install + tsx` 路线会装几十个依赖；我们用 Node 原生 `--experimental-transform-types` 直接跑。

✓ **预期结果：**打印 v24.x.x。旧版 Node 会报

`ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX`，升级即可。

实验 1 • 平衡仿真：9 种跑者各跑 5 局

操作：

```
cd F:\reminder\deadrun\exercise
node --experimental-transform-types simulate.ts
```

目的：用纯函数引擎 + 假 GPS 轨迹（"斥力逃跑"策略：远离 160m 内所有丧尸）headless 重放整局，验证难度经济。**验证：**每种画像应输出波次/存活时间/结局统计。

✓ **实测输出（节选）：**statue 3.6 波 2m16s caught=5 | runner 3.3m/s

14.4 波 10m33s 2085m | greedy 18.0 波 13m11s——站着必死、跑起来活得久、绕路捡补给活最久，设计意图全部兑现。

实验 2 • 投影校验：手搓地图 vs 教科书公式 + 真服务器

操作：

```
node --experimental-transform-types check-tiles.ts
```

目的： 31 项检查覆盖：4 个地标投影坐标 = 教科书 slippy-map 公式、100m 北 = 100m 像素、相机死区行为、朝向融合真值表、瓦片预算 ≤ 30 、真机下载瓦片。

✅ 实测：31/33 通过（投影/比例/预算/相机/朝向全绿）。

⚠️ 2 项 FAIL 是预期现象：没有 CARTO key 时每张瓦片被印上 "API KEY REQUIRED" 水印，市中心瓦片和太平洋瓦片变成同一大小的水印图，"城市比海洋细节多"无法用带水印图比较——不是投影错误。

实验 3 • 交叉验证：换 OSM 官方服务器证明坐标没错

操作：

```
node --experimental-transform-types osm-crosscheck.ts
```

目的： 实验 2 的水印干扰用对照组排除——把同一个 z/x/y 打给 OpenStreetMap 官方瓦片服务器，若坐标指错城市，市中心和公海会一样空白。

✅ 实测：Galata Tower z17/76085/49129 = 24835 B，太平洋同 zoom = 103 B，字节比 241 倍。坐标精确指向伊斯坦布尔真实街道，手搓投影宣告无罪。

实验 4 • 无头复放：把一整局追逐画成图

操作：

```
node --experimental-transform-types chase-demo.ts 300 42  
# 产出 chase-replay.svg，用浏览器打开即见整局追逐
```

目的：直接观察引擎行为——绿线是 3.3 m/s 玩家的逃跑轨迹，彩色细线是 41 只丧尸的追击轨迹，黄点是每波刷新点。**验证：**能看到丧尸轨迹向玩家轨迹收拢、玩家绕开尸群的急转弯。

✅ **实测：**seed 42 存活 5m00s • 7 波 • 990 m • 得分 6709；SVG 已渲染截图核实——W1 出生在起点旁，W3/W4/W5 的尸群轨迹呈包围收拢，玩家两次急转甩掉尾追。

🎁 这个项目给我什么

能力清单：① 不依赖 Google key 的手搓瓦片地图方案（CARTO/OSM + Web Mercator 三公式）；② GPS 健身类 App 的完整传感器融合套路（anchor 计步 / course-罗盘切换 / 反作弊闸门）；③ 纯函数游戏引擎架构（可仿真、可回放、可暂停时间旅行）；④ 波次制敌人生成与难度经济模板；⑤ 一套“细节为王”的工程文化范本——每个魔法数字都有注释解释为什么是这个值。

能拿它做什么：改造成“快递柜寻宝”AR 游戏、给自家小区做“定向越野 + NPC 追逐”团建工具、把引擎抽出来做任何“GPS 位置驱动”的小游戏、或把 check-tiles 式的“投影对照测试”抄进任何自绘地图项目。



知识入库要点

本项目为 vault 新增 4 个概念：**Web墨卡托与瓦片金字塔**（经纬度→z/x/y 编号瓦片，手搓地图的前提）、**相机死区跟随**（延迟跟随反而让状态可见的设计反直觉案例）、**GPS朝向融合**（course 管"去哪"/罗盘管"面向"，1.4 m/s 切换）、**纯函数游戏引擎与种子复放**（呼应 [[Checkpoint存档与持久执行]]）。关联已有概念：锚点计步 ↔ [[漂移与回环闭合]] 的"有界误差"思想；headless 仿真验收 ↔ [[证据优先质检 ProofOverClaims]]；纯函数引擎 ↔ [[确定性脚本]]。